

Week 1 Assignment

Python Fundamentals for Data Analytics

Student Name: ARPIT RAJ KATIYAR
College Name: Kanpur Institute of Technology
Branch: Computer Science Engineering (CSE-AIML)

Evaluation Criteria

Criteria	Marks
Python Basics	10
Variables & Data Types	10
Operators	10
Conditional Statements	10
Loops	10
Functions	10
Strings & Collections	15
File Handling	10
Mini Python Project	15
Total	100

Task 1: Python Basics (10 Marks)

Explanation

This program introduces basic input/output in Python. It prints a welcome message using `print()`, then uses `input()` to collect the user's Name, College Name, and Branch. The collected values are then displayed back in a clean, formatted layout using f-strings.

Python Source Code

```
"""
Task 1: Python Basics
-----
This program prints a welcome message, takes the user's Name, College Name,
and Branch as input, and displays the entered information in a formatted output.
"""

print("=" * 50)
print("WELCOME TO PYTHON FUNDAMENTALS FOR DATA ANALYTICS")
print("=" * 50)

# Taking user input
name = input("Enter your Name: ")
college = input("Enter your College Name: ")
branch = input("Enter your Branch: ")

# Displaying formatted output
print("\n----- Student Details -----")
print(f"Name           : {name}")
print(f"College Name : {college}")
print(f"Branch        : {branch}")
print("-" * 28)
print(f"Hello {name}! Wishing you a great learning journey in Data Analytics.")
```

Program Output (Sample Run)

```
=====
WELCOME TO PYTHON FUNDAMENTALS FOR DATA ANALYTICS
=====
Enter your Name: Enter your College Name: Enter your Branch:
----- Student Details -----
Name           : Aarav Sharma
College Name : ABC Institute of Technology
Branch        : Computer Science Engineering
-----
Hello Aarav Sharma! Wishing you a great learning journey in Data Analytics.
```

Task 2: Variables & Data Types (10 Marks)

Explanation

This program declares four variables representing the core primitive data types in Python: int, float, str, and bool. The built-in type() function is used to inspect and print the data type of each variable alongside its value.

Python Source Code

```
"""
Task 2: Variables & Data Types
-----
This program declares variables of different data types (int, float, string,
boolean) and prints each variable along with its data type using type().
"""

# Declaring variables of different data types
age = 21                # Integer
height = 5.9            # Float
student_name = "Aarav Sharma" # String
is_enrolled = True      # Boolean

# Printing variables with their data types
print("Value:", age, "\tData Type:", type(age))
print("Value:", height, "\tData Type:", type(height))
print("Value:", student_name, "\tData Type:", type(student_name))
print("Value:", is_enrolled, "\tData Type:", type(is_enrolled))
```

Program Output (Sample Run)

```
Value: 21 ■Data Type: <class 'int'>
Value: 5.9 ■Data Type: <class 'float'>
Value: Aarav Sharma ■Data Type: <class 'str'>
Value: True ■Data Type: <class 'bool'>
```

Task 3: Operators (10 Marks)

Explanation

This program implements a simple calculator. It accepts two numbers from the user and applies the arithmetic operators (+, -, *, /, %) to demonstrate Addition, Subtraction, Multiplication, Division, and Modulus operations. A check is added to avoid division/modulus errors when the second number is zero.

Python Source Code

```
"""
Task 3: Operators
-----
A simple calculator program that performs Addition, Subtraction,
Multiplication, Division, and Modulus using two numbers entered by the user.
"""

num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))

addition = num1 + num2
subtraction = num1 - num2
multiplication = num1 * num2
division = num1 / num2 if num2 != 0 else "Undefined (division by zero)"
modulus = num1 % num2 if num2 != 0 else "Undefined (division by zero)"

print("\n----- Calculator Results -----")
print(f"Addition      : {num1} + {num2} = {addition}")
print(f"Subtraction    : {num1} - {num2} = {subtraction}")
print(f"Multiplication  : {num1} * {num2} = {multiplication}")
print(f"Division        : {num1} / {num2} = {division}")
print(f"Modulus         : {num1} % {num2} = {modulus}")
```

Program Output (Sample Run)

```
Enter first number: Enter second number:
----- Calculator Results -----
Addition      : 20.0 + 4.0 = 24.0
Subtraction    : 20.0 - 4.0 = 16.0
Multiplication : 20.0 * 4.0 = 80.0
Division       : 20.0 / 4.0 = 5.0
Modulus        : 20.0 % 4.0 = 0.0
```

Task 4: Conditional Statements (10 Marks)

Explanation

This program demonstrates decision-making in Python using if, elif, and else. Based on the marks entered by the user, it assigns a grade: 90+ is Grade A, 75-89 is Grade B, 60-74 is Grade C, and anything below 60 is a Fail.

Python Source Code

```
"""
Task 4: Conditional Statements
-----
This program takes marks as input and displays the grade using
if, elif, and else statements.

Grading Scheme:
90+      -> A
75-89    -> B
60-74    -> C
Below 60 -> Fail
"""

marks = float(input("Enter your marks (out of 100): "))

if marks >= 90:
    grade = "A"
elif marks >= 75:
    grade = "B"
elif marks >= 60:
    grade = "C"
else:
    grade = "Fail"

print(f"\nMarks obtained : {marks}")
print(f"Grade           : {grade}")
```

Program Output (Sample Run)

```
Enter your marks (out of 100):
Marks obtained : 82.0
Grade          : B
```

Task 5: Loops (10 Marks)

Explanation

This program demonstrates the two main looping constructs in Python. A for loop with range() prints numbers 1 to 20 and also generates the multiplication table of a user-given number. A while loop is used separately to print all even numbers between 1 and 50.

Python Source Code

```
"""
Task 5: Loops
-----
This program demonstrates:
1. Printing numbers from 1 to 20 using a for loop.
2. Printing the multiplication table of a number entered by the user.
3. Printing even numbers from 1 to 50 using a while loop.
"""

# 1. Print numbers from 1 to 20 using a for loop
print("----- Numbers from 1 to 20 -----")
for i in range(1, 21):
    print(i, end=" ")
print("\n")

# 2. Multiplication table of a number
num = int(input("Enter a number to print its multiplication table: "))
print(f"\n----- Multiplication Table of {num} -----")
for i in range(1, 11):
    print(f"{num} x {i} = {num * i}")

# 3. Even numbers from 1 to 50 using a while loop
print("\n----- Even numbers from 1 to 50 (while loop) -----")
n = 1
while n <= 50:
    if n % 2 == 0:
        print(n, end=" ")
    n += 1
print()
```

Program Output (Sample Run)

```
----- Numbers from 1 to 20 -----
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20

Enter a number to print its multiplication table:
----- Multiplication Table of 7 -----
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70

----- Even numbers from 1 to 50 (while loop) -----
2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48 50
```

Task 6: Functions (10 Marks)

Explanation

This program defines two user-defined functions: `square(number)`, which returns the square of a number, and `average(a, b, c)`, which returns the mean of three numbers. Both functions are called with values entered by the user, demonstrating function definition, parameters, and return values.

Python Source Code

```
"""
Task 6: Functions
-----
Two user-defined functions:
1. square(number) -> calculates the square of a number.
2. average(a, b, c) -> calculates the average of three numbers.
Both functions are called using user input.
"""

def square(number):
    """Returns the square of the given number."""
    return number ** 2

def average(a, b, c):
    """Returns the average of three numbers."""
    return (a + b + c) / 3

# Calling square()
num = float(input("Enter a number to find its square: "))
print(f"Square of {num} = {square(num)}")

# Calling average()
n1 = float(input("\nEnter first number: "))
n2 = float(input("Enter second number: "))
n3 = float(input("Enter third number: "))
print(f"Average of {n1}, {n2}, {n3} = {average(n1, n2, n3)}")
```

Program Output (Sample Run)

Enter a number to find its square: Square of 9.0 = 81.0

Enter first number: Enter second number: Enter third number: Average of 4.0, 5.0, 6.0 = 5.0

Task 7: Strings & Collections (15 Marks)

Explanation

This program demonstrates key operations across Python's core data structures: string methods (upper, lower, replace, find), list methods (append, remove, sort), tuple creation and indexing/slicing, a dictionary used to store student information, and set operations (add, remove) to show uniqueness handling.

Python Source Code

```
"""
Task 7: Strings & Collections
-----
Demonstrates:
- String operations: upper(), lower(), replace(), find()
- List operations: append(), remove(), sort()
- Tuple creation and indexing
- Dictionary storing student information
- Set operations: add(), remove()
"""

# ----- String Operations -----
print("==== String Operations =====")
text = "Python for Data Analytics"
print("Original String :", text)
print("Uppercase       :", text.upper())
print("Lowercase       :", text.lower())
print("Replace         :", text.replace("Data Analytics", "Data Science"))
print("Find 'Data'      : index", text.find("Data"))

# ----- List Operations -----
print("\n==== List Operations =====")
fruits = ["Mango", "Apple", "Banana"]
print("Original List   :", fruits)

fruits.append("Orange")
print("After append() :", fruits)

fruits.remove("Banana")
print("After remove()  :", fruits)

fruits.sort()
print("After sort()    :", fruits)

# ----- Tuple Creation and Indexing -----
print("\n==== Tuple Operations =====")
coordinates = (10, 20, 30, 40)
print("Tuple           :", coordinates)
print("First element    :", coordinates[0])
print("Last element     :", coordinates[-1])
print("Slice [1:3]       :", coordinates[1:3])

# ----- Dictionary storing student information -----
print("\n==== Dictionary Operations =====")
student = {
    "name": "Aarav Sharma",
    "age": 21,
    "branch": "Computer Science",
    "cgpa": 8.7
}
print("Student Dictionary:", student)
for key, value in student.items():
    print(f"{key} : {value}")

# ----- Set Operations -----
print("\n==== Set Operations =====")
```

```
skills = {"Python", "SQL", "Excel"}
print("Original Set      :", skills)

skills.add("Power BI")
print("After add()       :", skills)

skills.remove("Excel")
print("After remove()    :", skills)
```

Program Output (Sample Run)

```
===== String Operations =====
Original String : Python for Data Analytics
Uppercase       : PYTHON FOR DATA ANALYTICS
Lowercase       : python for data analytics
Replace         : Python for Data Science
Find 'Data'     : index 11

===== List Operations =====
Original List   : ['Mango', 'Apple', 'Banana']
After append()  : ['Mango', 'Apple', 'Banana', 'Orange']
After remove()  : ['Mango', 'Apple', 'Orange']
After sort()    : ['Apple', 'Mango', 'Orange']

===== Tuple Operations =====
Tuple           : (10, 20, 30, 40)
First element   : 10
Last element    : 40
Slice [1:3]     : (20, 30)

===== Dictionary Operations =====
Student Dictionary: {'name': 'Aarav Sharma', 'age': 21, 'branch': 'Computer Science', 'cgpa': 8.7}
name : Aarav Sharma
age : 21
branch : Computer Science
cgpa : 8.7

===== Set Operations =====
Original Set     : {'Excel', 'Python', 'SQL'}
After add()      : {'Excel', 'Python', 'SQL', 'Power BI'}
After remove()   : {'Python', 'SQL', 'Power BI'}
```

Task 8: Basic File Handling (10 Marks)

Explanation

This program demonstrates basic file I/O in Python. It opens a file named introduction.txt in write mode ('w') and writes a short personal introduction into it, then reopens the same file in read mode ('r') to read and display its contents, confirming that the write operation succeeded.

Python Source Code

```
"""
Task 8: Basic File Handling
-----
This program creates a text file, writes an introduction into it,
and then reads and displays the file contents.
"""

file_name = "introduction.txt"

# Writing to the file
introduction = (
    "Hi, my name is Aarav Sharma. I am a Computer Science student "
    "learning Python for Data Analytics. I am passionate about "
    "exploring data, building models, and solving real-world problems."
)

with open(file_name, "w") as file:
    file.write(introduction)

print(f"'{file_name}' created and introduction written successfully.\n")

# Reading and displaying the file contents
with open(file_name, "r") as file:
    content = file.read()

print("----- File Contents -----")
print(content)
```

Program Output (Sample Run)

```
'introduction.txt' created and introduction written successfully.
```

```
----- File Contents -----
```

```
Hi, my name is Aarav Sharma. I am a Computer Science student learning Python for Data Analytics. I am passi
```

Task 9: Mini Python Project - Student Record Management System (15 Marks)

Explanation

This mini project ties together lists, dictionaries, functions, loops, and conditionals into a menu-driven console application. Each student is stored as a dictionary inside a list. The program supports four operations: adding a student, displaying all records, searching a student by name, and deleting a student record. The sample run below adds three students, displays them, searches for one, deletes another, and displays the updated list.

Python Source Code

```
"""
Task 9: Mini Python Project
-----
Student Record Management System

Features:
1. Add student details.
2. Display all student records.
3. Search a student by name.
4. Delete a student record.

Data is stored using a list of dictionaries.
"""

students = [] # Each student is stored as a dictionary

def add_student():
    name = input("Enter student name: ")
    roll_no = input("Enter roll number: ")
    branch = input("Enter branch: ")
    marks = input("Enter marks: ")

    student = {
        "name": name,
        "roll_no": roll_no,
        "branch": branch,
        "marks": marks
    }
    students.append(student)
    print(f"\nStudent '{name}' added successfully!\n")

def display_students():
    if not students:
        print("\nNo student records found.\n")
        return

    print("\n----- All Student Records -----")
    for i, student in enumerate(students, start=1):
        print(f"{i}. Name: {student['name']}, Roll No: {student['roll_no']}, "
              f"Branch: {student['branch']}, Marks: {student['marks']}")
    print()

def search_student():
    name = input("Enter the name of the student to search: ")
    found = False
    for student in students:
        if student["name"].lower() == name.lower():
            print(f"\nStudent Found -> Name: {student['name']}, "
```

```

        f"Roll No: {student['roll_no']], Branch: {student['branch']], "
        f"Marks: {student['marks']}\n")
    found = True
    break
if not found:
    print(f"\nNo student found with the name '{name}'.\n")

def delete_student():
    name = input("Enter the name of the student to delete: ")
    for student in students:
        if student["name"].lower() == name.lower():
            students.remove(student)
            print(f"\nStudent '{name}' deleted successfully!\n")
            return
    print(f"\nNo student found with the name '{name}'.\n")

def menu():
    while True:
        print("==== Student Record Management System =====")
        print("1. Add Student")
        print("2. Display All Students")
        print("3. Search Student by Name")
        print("4. Delete Student")
        print("5. Exit")

        choice = input("Enter your choice (1-5): ")

        if choice == "1":
            add_student()
        elif choice == "2":
            display_students()
        elif choice == "3":
            search_student()
        elif choice == "4":
            delete_student()
        elif choice == "5":
            print("Exiting the program. Goodbye!")
            break
        else:
            print("\nInvalid choice! Please enter a number between 1 and 5.\n")

if __name__ == "__main__":
    menu()

```

Program Output (Sample Run)

```

==== Student Record Management System =====
1. Add Student
2. Display All Students
3. Search Student by Name
4. Delete Student
5. Exit
Enter your choice (1-5): Enter student name: Enter roll number: Enter branch: Enter marks:
Student 'Aarav Sharma' added successfully!

==== Student Record Management System =====
1. Add Student
2. Display All Students
3. Search Student by Name
4. Delete Student
5. Exit
Enter your choice (1-5): Enter student name: Enter roll number: Enter branch: Enter marks:
Student 'Priya Verma' added successfully!

==== Student Record Management System =====

```

```
1. Add Student
2. Display All Students
3. Search Student by Name
4. Delete Student
5. Exit
Enter your choice (1-5): Enter student name: Enter roll number: Enter branch: Enter marks:
Student 'Rohan Gupta' added successfully!
```

```
===== Student Record Management System =====
1. Add Student
2. Display All Students
3. Search Student by Name
4. Delete Student
5. Exit
Enter your choice (1-5):
----- All Student Records -----
1. Name: Aarav Sharma, Roll No: 101, Branch: Computer Science, Marks: 88
2. Name: Priya Verma, Roll No: 102, Branch: Electronics, Marks: 91
3. Name: Rohan Gupta, Roll No: 103, Branch: Mechanical, Marks: 76
```

```
===== Student Record Management System =====
1. Add Student
2. Display All Students
3. Search Student by Name
4. Delete Student
5. Exit
Enter your choice (1-5): Enter the name of the student to search:
Student Found -> Name: Priya Verma, Roll No: 102, Branch: Electronics, Marks: 91
```

```
===== Student Record Management System =====
1. Add Student
2. Display All Students
3. Search Student by Name
4. Delete Student
5. Exit
Enter your choice (1-5): Enter the name of the student to delete:
Student 'Rohan Gupta' deleted successfully!
```

```
===== Student Record Management System =====
1. Add Student
2. Display All Students
3. Search Student by Name
4. Delete Student
5. Exit
Enter your choice (1-5):
----- All Student Records -----
1. Name: Aarav Sharma, Roll No: 101, Branch: Computer Science, Marks: 88
2. Name: Priya Verma, Roll No: 102, Branch: Electronics, Marks: 91
```

```
===== Student Record Management System =====
1. Add Student
2. Display All Students
3. Search Student by Name
4. Delete Student
5. Exit
Enter your choice (1-5): Exiting the program. Goodbye!
```